

# Plasticity Is All You Need? Persistent Low-Rank Fast Weights with Input-Subspace Protection

Thomas Nauheimer — September 2026 — preprint, work in progress

Status: exploratory results on one 0.5B-parameter model. The confirmatory multi-seed comparison against baselines is pending (Section 6.5). The title is a research question, not a result.

## Abstract

We study a frozen linear layer augmented with persistent low-rank fast weights  $A = UV^T$  whose updates are right-projected onto the orthogonal complement of a protected input subspace  $\text{span}(Q)$ . Updates are compressed by thin QR factorizations and a small core SVD, so the dense weight matrix is never formed. On synthetic regression, protected inputs are unchanged to machine precision ( $\leq 10^{-15}$ ) by construction. In a language model the layer targets are derived from the token loss, which turns the update into a projected, NLMS-normalized gradient step, i.e. test-time training of a low-rank state with gradient projection in the sense of GPM. On Qwen2.5-0.5B (24 plastic down\_proj layers, 48 fictional facts in 4 sessions) we find in exploratory runs that (i) most of the output drift caused by unprotected updates runs along the mean activation direction, so protecting a single direction removes a large part of it; (ii) forgetting across sessions is mainly interference, not rank truncation; and (iii) consolidating the fast weights at the end of each session and then growing  $Q$  from that session's activations reduces the forgetting of session 1 from about 3 nats to about 0. Consolidation is required: growing  $Q$  without it erases what was learned. The persistent state costs about 1.5 MB per learned fact (plus about 2.9 MB per fact for  $Q$  if learning is to continue), several orders of magnitude more than storing the fact as text for retrieval.

## 1. Related work

- **Fast weights.** Ba et al. (2016) used fast weights as temporary memory in recurrent networks; Schlag et al. (2021) showed that linear attention is a fast-weight programmer.
- **Differentiable plasticity.** Miconi et al. (2018) meta-learn plastic connection coefficients.
- **Gradient projection.** OWM (Zeng et al., 2019) and GPM (Saha et al., 2021) project updates onto the orthogonal complement of the input activations of earlier tasks. With gradient-derived targets (Section 2.2) our update is exactly this kind of projection, applied to a low-rank inference-time state instead of the base weights; we make no claim of novelty for the projection itself.
- **Low-rank continual learning.** O-LoRA (Wang et al., 2023) adds a LoRA adapter per task and penalizes overlap between the parameter subspaces of old and new adapters. InfLoRA (Liang & Li, 2024) constructs adapter subspaces that avoid interference with earlier tasks and is the closest prior work to the LoRA variant of our method (baseline B4). Like these methods, our procedure uses session boundaries: consolidation and subspace growth happen between sessions.
- **Test-time training.** TTT layers (Sun et al., 2024) and Titans (Behrouz et al., 2024) update hidden states by inner-loop gradient steps at inference time.
- **Massive activations.** Sun et al. (2024) report a few activation channels with values orders of magnitude above the median in LLMs; our drift analysis (Section 6.4) touches this phenomenon.
- **Retrieval.** A frozen model with retrieval of stored facts is a necessary baseline, not an impossible competitor.

## 2. Model and update

## 2.1 Layer

For row-batched inputs  $X \in \mathbb{R}^{N \times d_{\text{in}}}$ :

$$Y = X W^T + b + \gamma \cdot X (V_{\text{frozen}} U_{\text{frozen}}^T + V U^T)$$

$W$  is frozen.  $U, V \in \mathbb{R}^{d \times r}$  are the active fast weights;  $U_{\text{frozen}}, V_{\text{frozen}}$  hold consolidated sessions and are never updated. The gate  $\gamma \in [0, 1]$  scales all fast weights;  $\gamma = 0$  reproduces the base layer exactly.

Given target activations  $T$ , with  $E = T - Y$  and  $P = I - Q Q^T$ , one step is

$$A_{\text{cand}} = (1 - \lambda) A + (\eta \gamma / N) E^T X P$$

followed by truncation to rank  $r$  and a Frobenius-norm cap. This is the delta rule for the local squared error; it is stable only if  $\eta \cdot \lambda_{\text{max}}(X^T X / N) < 2$ .

## 2.2 Targets from the token loss

In a language model no layer targets exist. We use, per plastic layer,

$$T = Y - \ln r \cdot N / \text{mean}(\|x\|^2) \cdot \partial L / \partial Y$$

where  $L$  is the cross-entropy on the answer tokens. Then  $E \propto -\partial L / \partial Y$  and the step becomes

$$\Delta A = -\ln r \cdot (\partial L / \partial Y)^T X P / \text{mean}(\|x\|^2)$$

a projected gradient step on  $A$ , normalized as in NLMS. It requires a backward pass through the frozen network; it is not a forward-only update. The normalization makes the change of the layer output independent of the input scale of that layer.

## 3. Subspace protection and consolidation

If  $A_0 Q = 0$  and every update is right-projected by  $P$ , then  $A_t Q = 0$  for all  $t$ , so the layer output for inputs in  $\text{span}(Q)$  is unchanged by adaptation. This is an algebraic property of  $P$ , not an empirical finding. It protects only inputs inside  $\text{span}(Q)$ ; inputs with components outside  $\text{span}(Q)$  are changed, and changes propagate to later layers, whose inputs then leave their own calibrated subspaces. Layer invariance therefore does not imply model invariance; model-level drift has to be measured (Section 6).

**Consolidation.** To protect what a session has learned,  $Q$  is grown from the activations of that session. If the active fast weights still hold the learned mapping when  $Q$  grows, projecting them onto the new complement erases exactly the mapping one wanted to keep (observed: one further update with  $\ln r = 10^{-6}$  returned a learned fact to its pre-learning log-probability). Rank truncation, decay and the norm cap would also change  $A$  on  $\text{span}(Q)$ . The fast weights are therefore consolidated first: the active factors move into the frozen store and restart at zero, and only then is  $Q$  expanded. Since the active part starts at zero,  $A_{\text{active}} Q = 0$  holds for all later updates, and the frozen store is untouched. The implementation refuses to change  $Q$  while active factors are nonzero.

**Energy criterion.** New directions are added as in GPM: the residual  $R = X P$  of the session activations is decomposed by SVD, and leading directions are added until  $\text{span}(Q)$  holds a fraction  $\epsilon$  of the energy  $\|X\|_F^2$ .

## 4. Low-rank compression and cost

The candidate update is written as  $L R^T$  with  $m = r + N$  columns. Thin QR factorizations  $L = Q_L R_L$  and  $R = Q_R R_R$  reduce the SVD to the  $m \times m$  core  $R_L R_R^T$ . Cost per step:  $O((d_{\text{in}} + d_{\text{out}}) m^2 + m^3)$ ; forward overhead  $O(N r (d_{\text{in}} + d_{\text{out}}))$ ; storage  $O(r (d_{\text{in}} + d_{\text{out}}))$  per active or consolidated block plus  $O(k d_{\text{in}})$  for  $Q$ . The dense  $d_{\text{out}} \times d_{\text{in}}$  matrix is never formed. The store grows by  $r(d_{\text{in}} + d_{\text{out}})$  numbers per layer and session.

## 5. Synthetic verification

examples/reproduce\_benchmark.py: float64 on CPU, frozen affine map 32  $\rightarrow$  16, rank 8, 4 protected directions, 8 novel directions, 160 updates per seed; 20 held-out inputs are combinations of the novel directions, so the target correction lies within the available rank by construction.

| Seed | Held-out MSE before | after                  | Max change on span(Q) |
|------|---------------------|------------------------|-----------------------|
| 0    | 0.254999            | $2.74 \times 10^{-10}$ | $4.2 \times 10^{-17}$ |
| 1    | 0.291153            | $3.13 \times 10^{-10}$ | $4.2 \times 10^{-17}$ |
| 2    | 0.227038            | $2.44 \times 10^{-10}$ | $5.6 \times 10^{-17}$ |
| 3    | 0.413457            | $4.44 \times 10^{-10}$ | $5.9 \times 10^{-17}$ |
| 4    | 0.501408            | $5.38 \times 10^{-10}$ | $5.6 \times 10^{-17}$ |

The residual column depends on the platform BLAS; the MSE values reproduce exactly. This verifies the implementation, nothing more.

## 6. Language-model experiments (exploratory)

### 6.1 Setup

Qwen2.5-0.5B in float32; fast weights on down\_proj in all 24 layers ( $d_{in} = 4864$ ,  $d_{out} = 896$ ). 48 fictional facts (company  $\rightarrow$  headquarters, founder or product; names and answers generated per seed, so no pretrained knowledge applies) in 4 sessions of 12; the last session uses a different sentence template. 10 epochs per session, one fact per step. Metrics: exact top-1 recall of the answer span in a fresh context; the same for never-trained paraphrases; *forget S1*, the drop in session-1 answer log-probability between the end of session 1 and the end of training;  $KL(p_{base} \parallel p_{adapted})$  per token on held-out control text. Code: experiments/gradient\_targets\_drift.py.

### 6.2 Forgetting is mainly interference (seed 0, exploratory)

With a fixed Q of 16 directions from control text (rank 32), forget S1 is 4.09 nats without and 2.83 nats with per-session consolidation, and session-1 exact recall at the end is 0.17 in both cases. Rank truncation accounts for roughly a third of the forgetting; the rest is interference from later sessions. Raising the rank from 8 to 32 raises final recall ( $0.33 \rightarrow 0.52$  with consolidation) but does not restore retention.

### 6.3 Online subspace growth (seed 0, exploratory)

| Condition (rank 32)                                     | Recall | Paraphrase | Forget S1 (nats) | Ret. S1 exact | dim(Q)/ $d_{in}$ | KL held-out          |
|---------------------------------------------------------|--------|------------|------------------|---------------|------------------|----------------------|
| fixed Q ( $k = 16$ ) + consolidation                    | 0.52   | 0.38       | 2.80             | 0.17          | 0.003            | $6.7 \times 10^{-3}$ |
| online, $\epsilon = 0.97$ , base Q ( $\epsilon = 0.9$ ) | 0.69   | 0.48       | 0.00             | 0.83          | 0.060            | $1.8 \times 10^{-3}$ |
| online, $\epsilon = 0.97$ , no base Q                   | 0.85   | 0.56       | 0.01             | 0.83          | 0.027            | $1.4 \times 10^{-2}$ |

Growing Q from session activations after consolidation removes the measured forgetting in this setup. Protecting control-text directions as well (base Q) lowers drift by about a factor 8 and costs recall: the two variants span a trade-off, not a winner. Single seed, 48 facts; differences of  $\pm 0.1$  in recall correspond to a few facts.

### 6.4 Where the drift goes (exploratory)

Unprotected updates move the output distribution mostly along the mean input activation: with rank 32, protecting the single leading uncentered singular direction  $v_1$  lowers held-out KL from  $9.2 \times 10^{-2}$  to  $1.9 \times 10^{-2}$  (seed 0), while an SVD of mean-centered activations protects far less ( $2.3 \times 10^{-2}$  at  $k = 16$  versus  $7.9 \times 10^{-3}$  uncentered). In the middle layers  $\|\bar{x}\|^2$  exceeds the largest covariance eigenvalue by a factor 1.2–1.8 and  $|\cos(v_1, \bar{x})| \geq 0.93$ ; in layers 0 and 23 the ratio is below 1 and  $v_1$  is concentrated on a few very large channels (max/median channel magnitude up to 146 in layer 23).

On a smaller setup (12 facts, 3 seeds) the KL reduction relative to no protection was: protect  $\bar{x}/\|\bar{x}\|$  50 %,  $v_1$  33 %,  $v_1$  with its five largest channels removed 34 %, the five largest channels as coordinate directions 20 %, five median channels 10 %, five random channels –10 %. The spread of the two controls suggests noise of about  $\pm 10$  %, so the channel-level effect is suggestive rather than established, whereas the mean direction is clearly the larger lever. The size of the effect depends on the setup (30–80 % for  $v_1$ ). The analysis code for this subsection is not yet in the repository.

## 6.5 Baselines after tuning (seeds 100–102)

All methods use the same modules, facts, sessions, epochs and per-session parameter budget. Hyperparameters were selected per method on seeds 100–102 by final recall log-probability alone (five settings per method; learning rates for our method and for Adam-trained LoRA;  $\lambda$  for O-LoRA at the best B2 learning rate). The numbers below are therefore the maxima of a selection and optimistic; the confirmatory run on seeds 1–5 with the fixed settings is pending, as are the retrieval baselines on this model.

| Method                                  | Setting            | Recall | Para. | Forget S1 | Ret. S1 | KL held -out         | Appl  | MB/fact | Passes |
|-----------------------------------------|--------------------|--------|-------|-----------|---------|----------------------|-------|---------|--------|
| ours: online $\epsilon = 0.97$ , base Q | $lr = 1.0$         | 1.00   | 0.89  | 0.01      | 1.00    | $4.4 \times 10^{-2}$ | +0.28 | 1.47    | 480    |
| ours (next setting, for reference)      | $lr = 0.3$         | 0.99   | 0.86  | 0.01      | 1.00    | $7.7 \times 10^{-3}$ | +0.07 | 1.47    | 480    |
| B1 sequential LoRA                      | $3 \times 10^{-4}$ | 0.45   | 0.42  | 3.95      | 0.03    | $7.2 \times 10^{-1}$ | +8.7  | 0.37    | 480    |
| B2 LoRA per session, frozen             | $3 \times 10^{-4}$ | 0.53   | 0.49  | 4.09      | 0.06    | $7.5 \times 10^{-1}$ | +8.6  | 1.47    | 480    |
| B3 O-LoRA                               | $\lambda = 2.0$    | 0.58   | 0.50  | 3.68      | 0.11    | $7.9 \times 10^{-1}$ | +9.8  | 1.47    | 480    |
| B4 GPM-LoRA ( $A \leftarrow AP$ )       | $3 \times 10^{-4}$ | 0.83   | 0.71  | 1.33      | 0.47    | $4.1 \times 10^{-2}$ | +0.43 | 1.47    | 480    |
| B5 LoRA + replay                        | $3 \times 10^{-4}$ | 0.92   | 0.84  | 0.01      | 0.97    | $6.3 \times 10^{-1}$ | +6.7  | 0.37    | 840    |

Observations to be tested in the confirmatory run: (a) replay also removes forgetting but uses  $1.75\times$  the passes and drifts an order of magnitude more; (b) GPM-LoRA shares our protection guarantee and our Q, yet forgets more; one hypothesis is that Adam's per-coordinate scaling amplifies updates along the unprotected 3 % of session energy; (c) all LoRA variants drift strongly even at the smallest tuned learning rate; (d) the selected setting of our method drifts about  $6\times$  more than  $lr = 0.3$  for nearly the same recall, because selection used recall only; (e)  $\lambda = 2.0$  was at the upper edge of the O-LoRA grid, which has been extended. MB/fact excludes Q, which adds about 2.9 MB per fact for ours and B4 and is needed only to continue learning; retrieval stores about 100 bytes per fact.

## 7. Limitations and integration boundaries

- One model, one module type, synthetic facts with unique entity names, one template family plus one shifted template, 48 facts. Capacity limits over many sessions are untested; GPM predicts that Q fills up and later sessions learn less.
- The fast-weight store grows linearly with sessions; a data-weighted compression (minimizing  $\|(A - \hat{A}) X_{\text{sessions}}\|$  rather than  $\|A - \hat{A}\|$ ) is the natural next step.
- Learning needs backward passes. Modifying attention projections mid-sequence invalidates the KV cache; quantized or fused kernels need custom support for low-rank addends.
- Paraphrase generalization is measured only with templates close to the training question; reversal and multi-hop questions are untested.

## 8. Next experiments

- 1 Confirmatory run on seeds 1–5 with the fixed settings of Section 6.5, including retrieval and in-context baselines, with confidence intervals.
- 2 Capacity: 20–50 sessions and several hundred facts; trace  $\dim(Q)$  and per-session learning gain.
- 3 Store compression and its effect on forgetting.
- 4 Layer ablation: which layers carry the learned facts.
- 5 Generalization probes (different templates, reversal, two-hop questions) and targeted interference probes on related real-world knowledge.
- 6 A larger model with the settings fixed in advance.

## References

- Ba, J., Hinton, G., Mnih, V., Leibo, J. Z., Ionescu, C. (2016). Using fast weights to attend to the recent past. NeurIPS.
- Behrouz, A., Zhong, P., Mirrokni, V. (2024). Titans: Learning to memorize at test time. arXiv:2501.00663.
- Liang, Y.-S., Li, W.-J. (2024). InfLoRA: Interference-free low-rank adaptation for continual learning. CVPR.
- Miconi, T., Clune, J., Stanley, K. O. (2018). Differentiable plasticity. ICML.
- Saha, G., Garg, I., Roy, K. (2021). Gradient projection memory for continual learning. ICLR.
- Schlag, I., Irie, K., Schmidhuber, J. (2021). Linear transformers are secretly fast weight programmers. ICML.
- Sun, M., Chen, X., Kolter, J. Z., Liu, Z. (2024). Massive activations in large language models. COLM.
- Sun, Y. et al. (2024). Learning to (learn at test time): RNNs with expressive hidden states. arXiv:2407.04620.
- Wang, X. et al. (2023). Orthogonal subspace learning for language model continual learning. Findings of EMNLP.
- Zeng, G., Chen, Y., Cui, B., Yu, S. (2019). Continual learning of context-dependent processing in neural networks. Nature Machine Intelligence.